

NAME : P SAI VENKAT

ROLL NO : 2403A510G0

SUBJECT : AI ASSISTED CODING

ASSIGNMENT : 12.1

Task Description #1 (Sorting – Merge Sort Implementation)

- Task: Use AI to generate a Python program that implements the Merge Sort algorithm.

- Instructions:

- Prompt AI to create a function `merge_sort(arr)` that sorts a list in ascending order.

- Ask AI to include time complexity and space complexity in the function docstring.

- Verify the generated code with test cases.

- Expected Output:

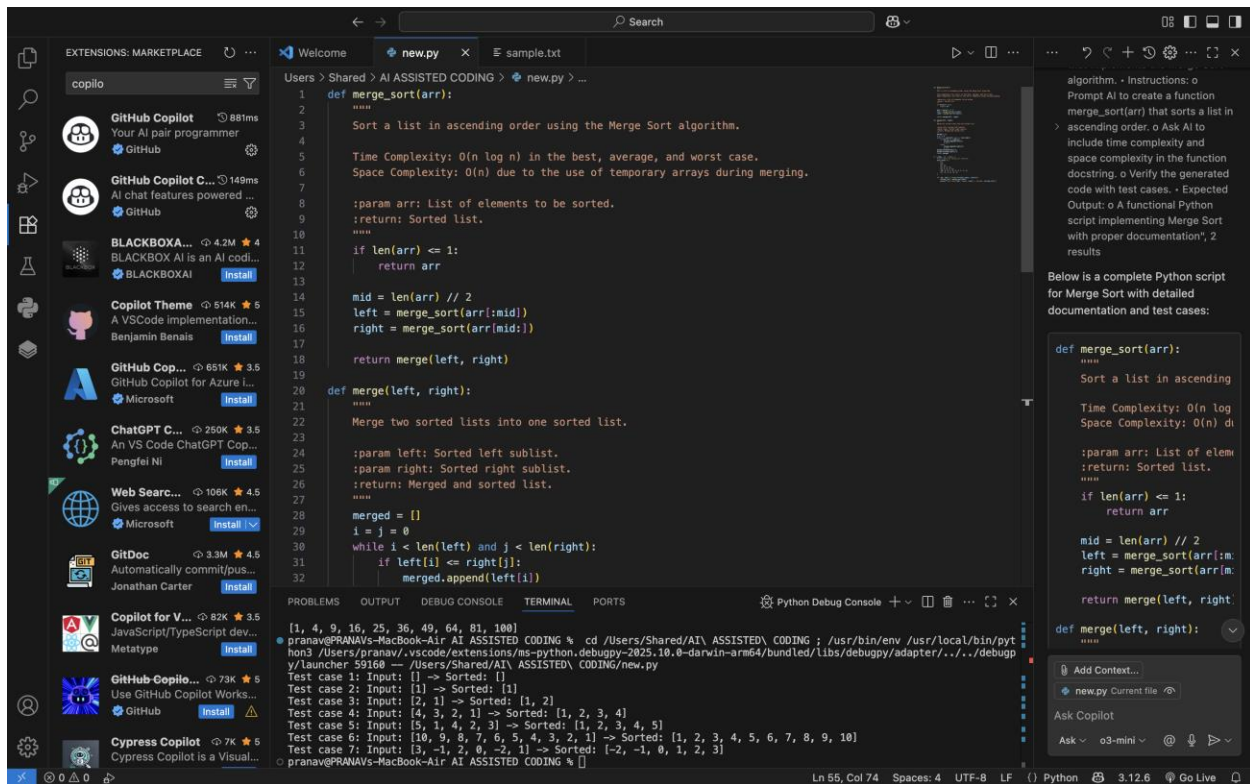
- A functional Python script implementing Merge Sort with proper documentation

PROMPT USED :

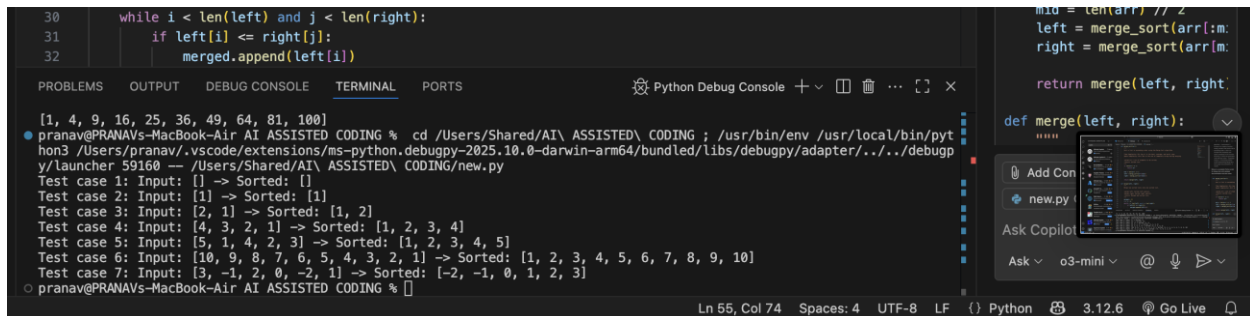
To generate a python program and to implement the merge sort algorithm

To sort in ascending order

CODE GENERATED :



OUTPUT :



Task Description #2 (Searching – Binary Search with AI Optimization)

- Task: Use AI to create a binary search function that finds a target element in a sorted list.
- Instructions:
 - Prompt AI to create a function `binary_search(arr, target)` returning the index of the target or -1 if not found.

- Include docstrings explaining best, average, and worst-case complexities.
- Test with various inputs.
- Expected Output:
- Python code implementing binary search with AI-generated comments and docstrings.

PROMPT USED :

To create a binary search function that finds a target element in a sorted list

CODE GENERATED :

The screenshot shows the VS Code editor with a file named `new.py` containing the following Python code:

```

1 def binary_search(arr, target):
2     """
3     Perform a binary search to find the index of a target element in a sorted list.
4
5     Best-case Time Complexity: O(1) when the target is found at the middle.
6     Average and Worst-case Time Complexity: O(log n).
7     Space Complexity: O(1) since this is an iterative implementation.
8
9     :param arr: A sorted list of elements.
10    :param target: The element to search for.
11    :return: The index of the target element if found, otherwise -1.
12    """
13    left, right = 0, len(arr) - 1
14
15    while left <= right:
16        mid = (left + right) // 2
17        if arr[mid] == target:
18            return mid
19        elif arr[mid] < target:
20            left = mid + 1
21        else:
22            right = mid - 1
23    return -1
24
25 if __name__ == "__main__":
26     # Test cases for binary_search function
27     sorted_list = [1, 3, 5, 7, 9, 11, 13, 15]
28     targets = [7, 1, 15, 8] # 8 is not present in the list
29
30     for target in targets:
31         index = binary_search(sorted_list, target)
32         if index != -1:
33             print(f"Target {target} found at index {index}.")
34         else:
35             print(f"Target {target} not found in the list.")

```

The right sidebar shows the AI-generated docstring and test cases, including the expected output for each test case.

OUTPUT :

The screenshot shows the VS Code editor with the output of the binary search function. The output is as follows:

```

Test case 6: Input: [10, 9, 8, 7, 6, 5, 4, 3, 2, 1] -> Sorted: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Test case 7: Input: [3, -1, 2, 0, -2, 1] -> Sorted: [-2, -1, 0, 1, 2, 3]
pranav@PRANAVS-MacBook-Air AI ASSISTED CODING % cd /Users/Shared/AI ASSISTED CODING ; /usr/bin/env /usr/local/bin/python3 /Users/pranav/.vscode/extensions/ms-python.debugpy-2025.10.0-darwin-arm64/bundled/libs/debugpy/adapter/../../debugpy/launcher 59187 -- /Users/Shared/AI ASSISTED CODING/new.py
Target 7 found at index 3.
Target 1 found at index 0.
Target 15 found at index 7.
Target 8 not found in the list.
pranav@PRANAVS-MacBook-Air AI ASSISTED CODING %

```

Task Description #3 (Real-Time Application – Inventory Management System)

- Scenario: A

retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.

- Task:

- o Use AI to suggest the most efficient search and sort

algorithms for this use case.

- o Implement the recommended algorithms in Python.

- o Justify the choice based on dataset size, update frequency, and performance requirements.

- Expected Output:

- o A table mapping operation → recommended algorithm → justification.

- o Working Python functions for searching and sorting the inventory.

Deliverables (For All Tasks)

1. AI-generated prompts for code and test case generation.
2. At least 3 assert test cases for each task.
3. AI-generated initial code and execution screenshots.
4. Analysis of whether code passes all tests.
5. Improved final version with inline comments and explanation.
6. Compiled report (Word/PDF) with prompts, test cases, assertions, code, and output.

PROMPT USED :

To write the code for a retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.

CODE GENERATED:

The screenshot shows the VS Code interface with the following components:

- Left Panel (EXTENSIONS: MARKETPLACE):** Lists various extensions including GitHub Copilot, BLACKBOX AI, Copilot Theme, ChatGPT Copilot, Web Search, GitDoc, Copilot for V..., GitHub Copilot..., and Cypress Copilot.
- Main Editor:** Displays a Python file named `new.py` with the following code:

```
1 def build_product_index(inventory):
2     """
3     Build an index (hash table) from product ID to product for quick lookup.
4
5     :param inventory: List of dictionaries representing products.
6     :return: Dictionary mapping product_id to product.
7     """
8     return {product['id']: product for product in inventory}
9
10
11 def search_by_id(inventory_index, product_id):
12     """
13     Retrieve product information using a pre-built inventory index.
14
15     :param inventory_index: Dictionary mapping product id to product.
16     :param product_id: The product ID to search for.
17     :return: The product dictionary if found; otherwise, None.
18     """
19     return inventory_index.get(product_id, None)
20
21
22 def search_by_name(inventory, name):
23     """
24     Search for products by name using a linear search.
25     This returns a list of matching products since names may not be unique.
26
27     :param inventory: List of product dictionaries.
28     :param name: The product name to search for.
29     :return: List of products with matching name.
30     """
31     return [product for product in inventory if product['name'] == name]
32
```
- Bottom Panel (TERMINAL):** Shows the output of test cases:

```
All search_by_id test cases passed.
All search_by_name test cases passed.
All sort_products_by_price test cases passed.
All sort_products_by_quantity test cases passed.

Sorted by Price:
{'id': 103, 'name': 'Doodad', 'price': 5.0, 'quantity': 200}
{'id': 102, 'name': 'Gadget', 'price': 15.75, 'quantity': 150}
{'id': 101, 'name': 'Widget', 'price': 25.5, 'quantity': 50}
{'id': 105, 'name': 'Widget', 'price': 25.5, 'quantity': 75}
{'id': 104, 'name': 'Thingamajig', 'price': 45.0, 'quantity': 10}
```
- Right Panel:** Contains a sidebar with a search bar and a list of products. Below it, a section titled "Below is the corresponding Python code:" shows the code for `build_product_index`, `search_by_id`, and `search_by_name`.

The screenshot shows the VS Code interface with the following components:

- Left Panel (EXTENSIONS: MARKETPLACE):** Same as the first screenshot.
- Main Editor:** Displays a Python file named `new.py` with the following code:

```
34 def sort_products_by_price(inventory):
35     """
36     :param inventory: List of product dictionaries.
37     :return: Sorted list of products.
38     """
39     return sorted(inventory, key=lambda product: product['price'])
40
41
42 def sort_products_by_quantity(inventory):
43     """
44     Sort the inventory by stock quantity in ascending order using Timsort.
45
46     :param inventory: List of product dictionaries.
47     :return: Sorted list of products.
48     """
49     return sorted(inventory, key=lambda product: product['quantity'])
50
51
52 if __name__ == "__main__":
53     # Sample inventory data representing a list of products.
54     inventory = [
55         {'id': 101, "name": "Widget", "price": 25.50, "quantity": 50},
56         {'id': 102, "name": "Gadget", "price": 15.75, "quantity": 150},
57         {'id': 103, "name": "Doodad", "price": 5.00, "quantity": 200},
58         {'id': 104, "name": "Thingamajig", "price": 45.00, "quantity": 10},
59         {'id': 105, "name": "Widget", "price": 25.50, "quantity": 75}
60     ]
61
62     # Build index for fast id-based search
63     inventory_index = build_product_index(inventory)
64
65     # Searching by ID test cases
66
```
- Bottom Panel (TERMINAL):** Shows the output of test cases:

```
All search_by_id test cases passed.
All search_by_name test cases passed.
All sort_products_by_price test cases passed.
All sort_products_by_quantity test cases passed.

Sorted by Price:
{'id': 103, 'name': 'Doodad', 'price': 5.0, 'quantity': 200}
{'id': 102, 'name': 'Gadget', 'price': 15.75, 'quantity': 150}
{'id': 101, 'name': 'Widget', 'price': 25.5, 'quantity': 50}
{'id': 105, 'name': 'Widget', 'price': 25.5, 'quantity': 75}
{'id': 104, 'name': 'Thingamajig', 'price': 45.0, 'quantity': 10}
```
- Right Panel:** Contains a sidebar with a search bar and a list of products. Below it, a section titled "Below is the corresponding Python code:" shows the code for `build_product_index`, `search_by_id`, and `search_by_name`.

OUTPUT :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  Python Debug Console  Python 3.12.6  Go Live

All search_by_id test cases passed.
All search_by_name test cases passed.
All sort_products_by_price test cases passed.
All sort_products_by_quantity test cases passed.

Sorted by Price:
{'id': 103, 'name': 'Doodad', 'price': 5.0, 'quantity': 200}
{'id': 102, 'name': 'Gadget', 'price': 15.75, 'quantity': 150}
{'id': 101, 'name': 'Widget', 'price': 25.5, 'quantity': 50}
{'id': 105, 'name': 'Widget', 'price': 25.5, 'quantity': 75}
{'id': 104, 'name': 'Thingamajig', 'price': 45.0, 'quantity': 10}

Ln 114, Col 23  Spaces: 4  UTF-8  LF  Python 3.12.6  Go Live

Sorted by Quantity:
{'id': 104, 'name': 'Thingamajig', 'price': 45.0, 'quantity': 10}
{'id': 101, 'name': 'Widget', 'price': 25.5, 'quantity': 50}
{'id': 105, 'name': 'Widget', 'price': 25.5, 'quantity': 75}
{'id': 102, 'name': 'Gadget', 'price': 15.75, 'quantity': 150}
{'id': 103, 'name': 'Doodad', 'price': 5.0, 'quantity': 200}
pranav@PRANAVs-MacBook-Air AI ASSISTED CODING %

Ln 114, Col 23  Spaces: 4  UTF-8  LF  Python 3.12.6  Go Live
```